

APACHE DORIS 3.0.8

复习题答案

第三章：数据导入、实时写入与一致性

15 道综合题参考答案。每题给出一句话结论、底层因果链、得分关键词和常见误区。

配套资料：notes-03

基准版本：Apache Doris 3.0.8

整理日期：2026-08

答案导航与评分

建议先口述完整答案，再对照本册。每题 10 分：结论 2 分，核心机制 4 分，因果链 2 分，边界或排障方法 2 分。

题号	核心主题	关键词
1	统一导入链路	Block、Tablet、Rowset、事务、Publish
2	Stream Load 重定向	FE 控制面、BE 数据面、307
3	Publish Timeout	COMMITTED、VISIBLE、同 Label
4	Broker Load	异步 Job、并行拉取、文件形态
5	Routine Load 三级模型	Job、Task、Transaction
6	Kafka Exactly Once	Offset 与事务共同推进
7	高频小批	事务、版本、小 Rowset、Compaction
8	Group Commit 模式	sync、async、WAL、可见性
9	Group Commit 限制	Label、2PC、共享事务
10	三层一致性	原子性、幂等、端到端 EOS
11	Unique 不等于 EOS	结果覆盖、重复投递、表模型
12	整行与部分列	NULL/default、partial_columns
13	部分更新代价	整行补齐、读写放大、3.0.8
14	Sequence	业务版本、乱序、旧事件晚到
15	故障排查	Filtered、Publish、Offset

1. 为什么所有导入方式最终都可以用 Rowset、事务与 Publish 解释？

导入方式只决定数据从哪里来、谁驱动以及任务生命周期；进入 Doris 存储层后，都需要解析成列式数据、路由到 Tablet、多副本写出 Rowset，再由事务提交和版本发布控制可见性。

入口不同，存储与事务不重新发明



Stream Load 由客户端 Push；Broker Load 由 BE 拉远程文件；Routine Load 由常驻 Job 消费 Kafka；Flink Connector 由外部 Checkpoint 驱动。但目标都不是简单复制源文件，而是产生符合 Doris 排序、编码、表模型和分桶规则的新 Segment 与 Rowset。

得分关键词

入口差异；解析 Block；分区分桶；Tablet 多副本；Pending Rowset；COMMITTED；VISIBLE

常见误区：把 Broker Load 理解成把 Parquet 原封不动拷进 Doris。源文件只是输入，Doris 会重新编码并组织为自己的 Segment。

2. Stream Load 为什么由 FE 重定向到 BE？客户端网络需要满足什么条件？

FE 负责认证、元数据、事务和选择协调节点，主体数据由协调 BE 接收并分发，避免 FE 成为大流量瓶颈。客户端必须跟随 307，并能访问 FE 返回的 BE 地址。

因果链

- 客户端先请求 FE 的标准入口。
- FE 选择协调 BE 并返回 HTTP 307。
- curl 使用 --location-trusted 跟随重定向并继续携带认证。
- 协调 BE 解析、计算分区分桶并向目标副本写入。
- 若客户端只能访问 FE 而不能访问 BE，请求会在重定向后失败。

得分关键词

FE 控制面；BE 数据面；307；--location-trusted；客户端到 BE 可达；避免 FE 带宽瓶颈

常见误区：只验证 client → FE 网络。Stream Load 还需要 client → coordinator BE 的数据通路。

3. Publish Timeout 与普通失败有什么本质区别？

Publish Timeout 通常表示事务已经 COMMITTED，只是未在等待窗口内完成版本发布；普通 Fail 表示导入没有成功提交。前者不能换新 Label 盲目重发。

```
PREPARE -> COMMITTED -> VISIBLE
          ^
          Publish Timeout
```

正确处理是记录 Label 与 TxnId，通过 SHOW TRANSACTION 查看状态。COMMITTED 继续等待发布，VISIBLE 按成功处理，ABORTED 才在修复原因后重试。新 Label 重发会创建新的业务事务，可能重复数据。

得分关键词

COMMITTED 已可靠提交；VISIBLE 才可读；SHOW TRANSACTION；同 Label；禁止盲目重发

常见误区：把超时等同于失败。分布式系统里的超时经常表示调用方不知道结果，而不是服务端一定没完成。

4. 为什么 Broker Load 适合大规模历史回灌？文件过大或过小各有什么问题？

Broker Load 把有限大批作业交给 FE 持久化管理，让多个 BE 直接并行读取 HDFS/S3；客户端无需中转数据或保持长连接。

文件形态	后果	优化
单个巨大且不可切分	Scanner 并发不足，集群吃不饱	合理拆分或使用可并行列式格式
海量极小文件	枚举、打开、请求和调度固定开销过大	上游合并文件
单个超大 Job	失败面、事务时间、内存与重试代价大	按日期或业务范围拆 Label

得分关键词

异步 Job；BE 直接拉取；Scan Range；并行；客户端断开不影响；文件大小平衡；按批拆事务

常见误区：认为文件越多越快。极小文件会让元数据与对象存储请求开销超过有效扫描。

5. Routine Load Job、Task 和 Transaction 的生命周期分别是什么？

Job 长期存在并管理源与进度；Task 是一次有限消费执行；每个 Task 对应一个短事务，完成后推进 Offset 并生成下一轮 Task。

对象	生命周期	职责
Job	长期运行	保存 Topic、目标表、参数、消费进度和状态
Task	数秒到一个微批	在 BE 上消费指定 Kafka Partition 范围
Transaction	随 Task 完成	让本批 Rowset 与 Offset 结果原子可恢复

无限数据流不能放在一个永远不提交的事务中，否则数据永远不可见、失败面无限增长。因此 Routine Load 用一系列有限事务表达持续流。

得分关键词

Job 常驻；Task 微批；Task 对应事务；有限事务循环；Offset 进度；自动调度恢复

常见误区：把 Routine Load Job 当成一个无限长事务。它是不断生产事务的调度器。

6. Routine Load 如何让 Kafka Offset 与 Doris 数据不丢不重？

一个 Task 将所消费 Offset 区间与 Doris 事务结果绑定：事务失败则数据不可见且 Offset 不推进；成功则数据提交并持久化下一消费位置。

避免先推进 Offset 导致丢失，也避免先写数据后丢进度导致重复



Exactly Once 的边界是 Routine Load 管理的 Kafka → Doris 链路。若 Kafka retention 已物理删除旧 Offset，任何事务协议都不能凭空找回，需要历史补数或接受跳过。

得分关键词

Offset 区间；事务提交；失败不推进；重读安全；进度持久化；retention 边界

常见误区：只依赖 Kafka consumer group 当前 Offset。Doris 还要让自己的事务结果与消费进度一起恢复。

7. 高频小批为什么会同时伤害 FE、Tablet 版本和 Compaction？

每个小写都重复支付解析、计划、事务、Publish 和 Rowset 固定成本；版本增长让查询打开更多 Rowset，并迫使 Compaction 高频重写。

数据量不大，也可能因为写得过碎形成恶性循环



优先在客户端形成微批；多客户端无法有效攒批时再用 Group Commit；同时检查表分桶、写入分区范围和 Compaction 资源。

得分关键词

固定开销；FE 事务；版本；小 Rowset；读放大；Compaction 写放大；客户端微批；Group Commit

常见误区：只调高并发。并发可能更快地制造版本和 Rowset，使根因恶化。

8. Group Commit sync_mode 与 async_mode 成功返回时分别意味着什么？

sync_mode 等待共享事务提交并可见后返回；async_mode 在数据写入接收 BE 的本地 WAL 后快速返回，合并事务和可见性发生在之后。

维度	sync_mode	async_mode
返回条件	合并事务提交完成	本地 WAL 持久化
立即查询	可以	不保证
客户端延迟	受合并窗口影响	更低
主要风险	等待时间与版本窗口	WAL 磁盘、单副本提前确认
得分关键词	sync 返回后可见；async 仅 WAL；后台提交；可见延迟；WAL 监控；Decommission	

常见误区：看到 async Stream Load Status=Success 就断言事务已经 VISIBLE。

9. 为什么用户 Label、Stream Load 2PC 与真正的 Group Commit 存在冲突？

Group Commit 要让多个请求共享一个事务身份；用户 Label 要求本请求拥有独立事务身份；2PC 又要求一个外部协调者能单独决定该事务 commit 或 abort，三者不能同时成立。

若把 A、B、C 三个用户请求合成事务 T，用户 A 就不能单独 abort T，因为会连带 B、C；也无法同时让 T 拥有三个彼此独立的 Label。因此 Doris 3.0 对指定 Label、2PC、部分列更新等请求会退化为普通独立导入。

得分关键词	共享事务；独立身份冲突；外部 commit/abort 控制；退化普通导入；逐请求幂等边界
常见误区：	同时设置 group_commit、label、two_phase_commit 后假定三个机制都生效。必须检查返回结果和文档限制。

10. 事务原子性、Label 幂等和端到端 Exactly Once 分别解决什么？

原子性解决单批内部不撕裂；Label 解决同一批次重试只成功一次；端到端 EOS 解决上游进度与 Doris 事务一起恢复。

层次	保证	不能单独解决
事务原子性	同批所有目标 Tablet 整体提交或回滚	同一成功批次被再次提交
Label 幂等	相同稳定批次在去重窗口内只成功一次	外部 Offset/Checkpoint 何时推进
2PC / Offset 协调	外部进度与 Doris 事务同一逻辑完成点	源数据已被物理删除
得分关键词	原子性；稳定批次；Label 去重窗口；Checkpoint / Offset；commit/abort；端到端边界	

常见误区：用一句“Doris 支持事务”概括全部 EOS。事务只是必要条件，不自动包含外部源状态。

11. Unique Key 为什么不能替代 Exactly Once 协议？

Unique Key 只让相同主键最终呈现一个结果，并不证明消息只产生一次业务效果，也不保护 Aggregate、Duplicate、乱序、审计和写入成本。

- Duplicate Key 会真实保留两条重复事件。
- Aggregate Key 的 SUM 可能把重复消息累计两次。
- Unique Key 也可能被旧事件晚到覆盖，需要 Sequence。
- 重复写仍产生事务、Rowset、Delete Bitmap 和 Compaction 成本。
- 若一条消息还有外部副作用，主键覆盖无法回滚或去重那些副作用。

得分关键词

结果层幂等；不等于传输 EOS；Aggregate 重复累计；Duplicate 保留；Sequence；存储成本

常见误区：SELECT 最终只有一行就宣称链路 Exactly Once。那只是当前查询结果的覆盖效果。

12. 整行 UPSERT 与部分列更新有什么危险差异？

Unique Key 默认整行 UPSERT；仅列出部分字段但未启用 partial update 时，未提供列会用 NULL 或默认值补齐并覆盖旧值。部分列更新才会保留旧行的未提供字段。

old: id=1, user_id=100, amount=88.50, status=created
input: id=1, status=paid

whole-row upsert -> user_id/amount may become NULL or default
partial update -> user_id=100, amount=88.50 remain

Stream Load 使用 partial_columns:true 且 columns 必须包含全部 Key；INSERT 需要 enable_unique_key_partial_update=true。

得分关键词

默认整行；缺失列补 NULL/default；partial_columns；全部 Key；INSERT session variable

常见误区：把 SQL 中只写两列视为数据库自动保留其他列。Doris 为防止语义歧义默认仍是整行 UPSERT。

13. 部分列更新为什么会产生读写放大，3.0.8 有什么批次限制？

MOW 为保持查询期只读最终完整行，需要在写入时读取旧行缺失列、补成完整行并写出新 Rowset；3.0.8 同一批所有行必须更新相同列集合。

宽表少列更新会产生明显读写放大



高频宽表部分更新适合高速 SSD；store_row_column 用额外存储换取一次读取整行表示。3.0.8 不支持同一 JSON 批中每行更新不同列的 Flexible Partial Update，该能力从 3.1.0 才出现。

得分关键词

读旧行；整行补齐；写新完整行；IOPS；store_row_column；同批同列；3.1 才灵活列更新

常见误区：以为 partial_columns:true 会让物理文件只新增提供的几列。MOW 最终仍要形成查询高效的完整行版本。

14. Sequence Column 怎样阻止旧 CDC 事件覆盖新状态？

相同 Unique Key 下以业务 Sequence 比较新旧：较大值可以替换较小值，旧事件即使晚到也不会让状态倒退。

current: order_id=1, status=paid, sequence=20

late: order_id=1, status=created, sequence=10

10 < 20 -> late row cannot replace current row

Sequence 可用业务版本、binlog LSN、单调更新时间；同值时应有额外稳定规则。Kafka 只保证单 Partition 顺序，跨 Partition 或多并发导入仍可能乱序。删除事件也应携带 Sequence，防止旧更新复活已删除数据。

得分关键词

相同 Key；业务版本；大值胜出；旧事件晚到；LSN；删除也排序；避免同值

常见误区：用 Doris 接收时间作 Sequence。网络慢的旧事件可能得到更晚接收时间，反而覆盖正确新状态。

15. 遇到 too many filtered rows、Publish Timeout 和 Offset out of range 应分别怎样处理？

现象	含义	处理
too many filtered rows	质量错误比例超过阈值	核对行数并看 ErrorURL，修复格式、类型、列映射或分区；不先调到 1
Publish Timeout	通常 COMMITTED 未及时 VISIBLE	记录 Label/TxnId，SHOW TRANSACTION；不换新 Label 重发
Offset out of range	Kafka 已清理 Doris 记录位置	判断历史补数或接受跳过，再重置到有效 Offset；调整 retention

五层排查框架

- 入口：认证、307、FE 与 BE 网络、请求大小和超时。
- 质量：CSV/JSON 格式、类型、精度、列数、strict_mode、ErrorURL。
- 路由存储：分区、时区、Tablet 副本、磁盘、内存、热点和版本数。
- 事务发布：Label、PREPARE、COMMITTED、VISIBLE、ABORTED。
- 持续任务：State、Progress、ReasonOfStateChanged、Lag、retention。

得分关键词

ErrorURL；不盲目 max_filter_ratio=1；COMMITTED 非失败；同 Label；retention 物理边界；分层排查

常见误区：对所有错误统一重试。系统背压和版本过多时，重试会继续制造压力；语义错误重试也不会自动变正确。

总复盘

第三章的主线不是记住六种导入命令，而是判断数据入口、批次边界、表模型、事务身份、外部进度和存储整理成本是否形成闭环。

总分	掌握程度	建议
130-150	能从业务语义推导入口与一致性方案	进入查询执行与优化章节
100-129	主体清楚，部分更新或故障边界不牢	重画 2PC、Routine Offset、MOW 三张图
70-99	会用命令但原理链不完整	重看 3.2、3.4、3.5、3.6
0-69	容易把原子性、幂等和 EOS 混用	从统一 Rowset 链路重新学习

15 题一句话答案

题号	一句话
1	入口不同，但最终都路由 Tablet、写 Rowset、提交并发布版本。
2	FE 只做控制，数据进 BE；客户端必须跟随 307 且可访问 BE。
3	Publish Timeout 多为已提交未可见，应查原事务而非换 Label 重发。
4	Broker Load 让 BE 并行拉远程文件；文件大小要兼顾并发和元数据成本。
5	Job 常驻、Task 有限、每 Task 一个短事务并循环推进。
6	数据事务成功才推进 Offset，失败则数据不可见且从原位置重读。
7	每个小写都制造事务、版本和 Rowset，最终压垮 Compaction。
8	sync 返回后可见；async 只保证先写本地 WAL，稍后可见。
9	共享事务无法同时给每个请求独立 Label 或独立 2PC 决策权。
10	原子性管单批，Label 管重试，2PC/Offset 管外部进度。
11	Unique 只是结果覆盖，不是传输 Exactly Once。
12	未开部分更新时，缺失列会用 NULL/default 做整行覆盖。
13	部分更新要读旧值补整行再写；3.0.8 同批只能更新相同列。
14	Sequence 让业务版本大的状态胜出，抵抗旧事件迟到。
15	脏数据看 ErrorURL，发布超时查事务，Offset 越界先决定补数。

参考资料

答案依据第三章教程，并以 Apache Doris 3.0 与 3.x 官方文档中的稳定机制校验。

[1] Stream Load

<https://doris.apache.org/docs/3.0/data-operate/import/import-way/stream-load-manual/>

[2] Broker Load

<https://doris.apache.org/docs/3.0/data-operate/import/import-way/broker-load-manual/>

- [3] Routine Load
<https://doris.apache.org/docs/3.0/data-operate/import/import-way/routine-load-manual/>
- [4] Group Commit
<https://doris.apache.org/zh-CN/docs/3.0/data-operate/import/group-commit-manual/>
- [5] Transaction
<https://doris.apache.org/docs/3.0/data-operate/transaction/>
- [6] Partial Column Update
<https://doris.apache.org/docs/3.0/data-operate/update/partial-column-update/>
- [7] Unique Update Concurrent Control
<https://doris.apache.org/docs/3.0/data-operate/update/unique-update-concurrent-control/>
- [8] Handling Messy Data
<https://doris.apache.org/docs/3.x/data-operate/import/handling-messy-data/>